



POLITECHNIKA OPOLSKA

PROGRAMOWANIE SYSTEMOWE

Pamięć współdzielona i wzajemne wykluczanie.

Ćwiczenie Nr 4

Proszę napisać program wykorzystujący elementarne operacje Uniksa/QNXa zarządzania procesami, tworzenia pamięci współdzielonej oraz semaforami.

Zadanie 4.1:

Proszę napisać program symulujący opisaną poniżej sytuację:

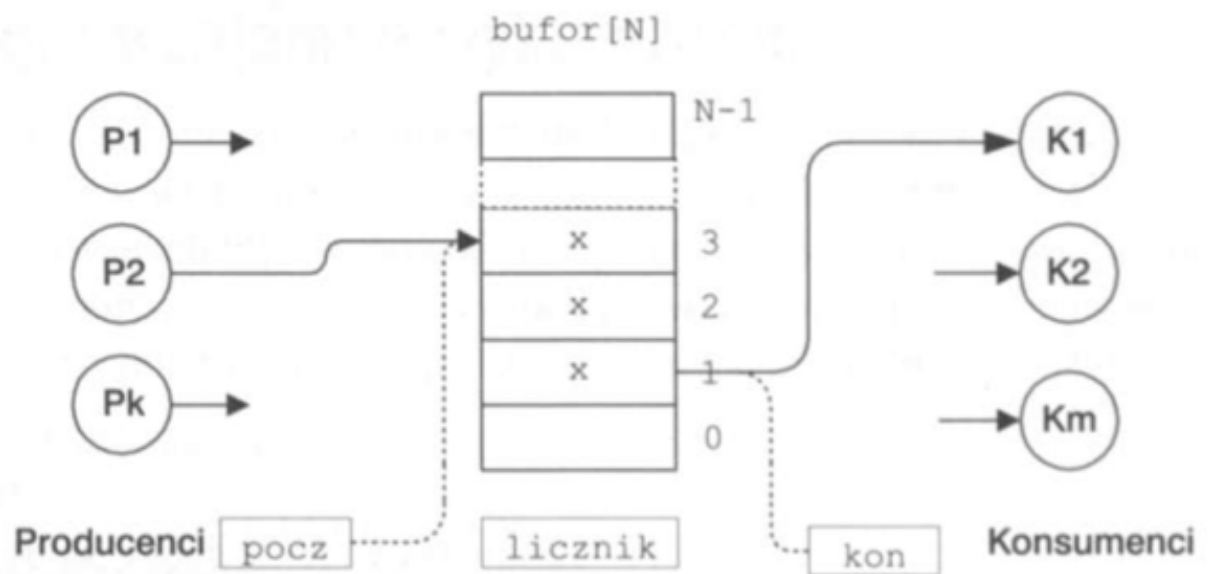
- Pszczoły zbierają miód i zanoszą do ula. Misie podbierają miód z ula co jakiś czas.
- Ilość miodu znajdującego się w ulu powinna być reprezentowana przez wartość w pamięci współdzielonej.
- Pszczoły: produkują miód (zwiększają wartość zmiennej współdzielonej o losową wartość. Każda pszczoła posiada własny licznik określający, ile zniosła do ula od początku.
- Misie: zjadają miód (zmniejszają wartość zmiennej współdzielonej o losową wartość. Miś nie może zjeść więcej miodu niż jest w ulu. Każdy miś posiada własny licznik określający, ile zjadł z ula od początku.

1. Program powinien uruchamiać się z dwoma argumentami: pierwszy liczba pszczół, drugi - liczba misiów.
2. Proces macierzysty powinien uruchomić odpowiednią liczbę procesów „pszczoła” oraz „miś”, stworzyć pamięć współdzieloną, utworzyć plik z danymi wynikowymi oraz semafor dostępu na wyłączność w czasie dostępu do pamięci współdzielonej i pliku.
3. Każdy proces powinien wylosować liczbę odwiedzin w ulu (np. `rand()%100`) dla pszczół i `rand()%10` dla misiów.
4. Przy każdych odwiedzinach proces powinien wylosować masę miodu do dodania/zabrania (np. `rand()%10` dla pszczoły i `rand()%50` dla misia). Po każdym dodaniu / zmniejszeniu powinien być wygenerowany komunikat i zapisany jako kolejna linia w pliku, np:
Pszczoła_X: zniosłam 3g miodu, w ulu obecnie jest 342g miodu.
Miś_X: zjadłem 10g miodu, w ulu obecnie jest 332g miodu.
Gdzie X to PID procesu.
5. Po zapisaniu komunikatu każdy proces pszczoły czy misia usypia na losowy czas (np. 0-100 ms dla pszczoły, 0-1000 ms dla misia).
6. Proces macierzysty czeka na zakończenie procesów potomnych.

Zadanie 4.2:

Proszę napisać program symulujący opisaną poniżej sytuację:

- Producent produkuje paczkę danych i jeśli w buforze jest wolne miejsce to umieszcza ją w buforze.
- Jeśli w buforze są jakieś paczki danych to Konsument pobiera jedną paczkę danych z bufora.
- Bufor znajduje się w pamięci współdzielonej; jest buforem cyklicznym i ma ograniczoną pojemność np. na 10 paczek danych.
- **program powinien korzystać wyłącznie z semaforów.**



1. Program powinien być uruchamiany z dwoma argumentami - pierwszy - liczba procesów „Producent”, drugi - liczba procesów „Konsument”.
2. Proces macierzysty powinien uruchomić odpowiednią liczbę procesów Konsument oraz Producent, stworzyć pamięć współdzieloną oraz odpowiednie semafony.
3. Jeśli w buforze jest wolne miejsce proces Producent może wpisać do bufora paczkę danych, a następnie wygenerować komunikat na ekranie:
Producent_X: dane wpisane: Time= 22.30 Temp=23.3, Hum = 19%
i zapisać do pliku w nowej linii:
Producent_X umieścił produkt, ilość elementów w buforze: 3
4. Jeśli w buforze są dane to proces Konsument może pobrać z bufora paczkę danych, a następnie wygenerować komunikat na ekranie:
Konsument_X dane pobrane: Time= 21.00 Temp=27.3, Hum = 17%
i zapisać do pliku w nowej linii:

- Konsument X pobrał produkt, ilość elementów w buforze: 2
5. Po zapisaniu komunikatu każdy procesów usypia się na losowy czas dla (np. 0 - 1000 ms).
 6. Proces macierzysty oczekuje na dane z klawiatury. Jeśli naciśnięto wybrany klawisz (np. 'q') proces powinien wpisać informację o końcu programu do pamięci współdzielonej.
 - 6.1 Wszystkie procesy Producentów powinny się zakończyć po wykryciu rozkazu zamknięcia procesów.
 - 6.2 Wszystkie procesy Konsumentów powinny się zakończyć po pobraniu wszystkich elementów z bufora.
 - 6.3 Proces macierzysty powinien zamknąć plik i zakończyć się.

Przykładowa paczka danych może wyglądać następująco

```
Typedef struct{
    int  godzina;           // wartość 0000 - 2359
    int  PID_stacji;        // np. PID procesu
    float temperatura;      //
    int  wilgotosc;         // wartość 0 - 100
} dane_t;
```

Zadanie 4.3:

Proszę zmodyfikować program z zadania 4.2 tak, aby korzystał z muteksów oraz zmiennych warunkowych.

przydatne funkcje:

<https://linux.die.net/>

https://man7.org/linux/man-pages/dir_all_alphabetic.html

```
#include <fcntl.h>
int open(const char *pathname, int flags, mode_t mode);
int creat(const char *pathname, mode_t mode);

#include <unistd.h>
int access(const char *pathname, int mode);
int close(int fd);
int unlink(const char *pathname);
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);

pid_t fork(void);
int execl(const char *path, const char *arg, ..., argn, NULL);
pid_t getpid(void);
pid_t getppid(void);

int usleep(useconds_t usec);
int ftruncate(int fildes, off_t length);

#include <sys/wait.h>
pid_t wait(int* wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);

// pamięć współdzielona
#include <sys/mman.h>
#include <sys/stat.h>          /* For mode constants */
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
void *mmap(void addr[.length], size_t length, int prot, int
flags, int fd, off_t offset);

#include <sys/shm.h>
int shmget(key_t key, size_t size, int shmflg);
void* shmat(int shmid, const void *shmaddr, int shmflg);
int shmdt(const void *shmaddr);
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

```

// semafor
#include <sys/stat.h>
#include <semaphore.h>
sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned
int value);
int sem_close(sem_t *sem);
int sem_unlink(const char *name);

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);

int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_timedwait(sem_t *restrict sem, const struct timespec
*restrict abs_timeout);
int sem_post(sem_t *sem);
int sem_getvalue(sem_t *restrict sem, int *restrict sval);

#include <sys/sem.h>
int semget(key_t key, int nsems, int semflg);
int semctl(int semid, int semnum, int cmd, ...);
int semop(int semid, struct sembuf *sops, size_t nsops);

// muteksy
#include <pthread.h>
int pthread_mutexattr_init(pthread_mutexattr_t *attr);
int pthread_mutexattr_destroy(pthread_mutexattr_t *attr);

int pthread_mutexattr_getpshared(const pthread_mutexattr_t
*restrict attr, int *restrict pshared);
int pthread_mutexattr_setpshared(pthread_mutexattr_t *attr, int
pshared);

int pthread_mutex_init(pthread_mutex_t *restrict mutex, const
pthread_mutexattr_t *restrict attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);

```

```
// zmienne warunkowe
int pthread_cond_init(pthread_cond_t *restrict cond,
                      const pthread_condattr_t *restrict attr);
int pthread_cond_wait(pthread_cond_t *restrict cond,
                      pthread_mutex_t *restrict mutex);
int pthread_cond_timedwait(pthread_cond_t *restrict cond,
                           pthread_mutex_t *restrict mutex,
                           const struct timespec *restrict abstime);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_destroy(pthread_cond_t *cond);
```